

1. Introduction

Reinforcement Learning (RL) is a branch of machine learning in which an agent learns to make decisions by interacting with an environment through trial and error. Rather than being given correct answers directly, the agent takes actions, receives feedback in the form of rewards, and gradually improves its behavior based on that feedback. Frozen Lake is a classic grid-world problem used to study RL. An agent must navigate an 8×8 grid from a Start state to a Goal state while avoiding Holes scattered across the ice. The agent has no prior knowledge of where the holes are and must learn a safe path purely through repeated experience. This report documents a complete from-scratch implementation of the Frozen Lake environment and a Q-Learning agent, without using any existing RL frameworks.

2. Environment Design

State and Action Representation

The grid is 8×8 , giving 64 possible states. Each state is represented as a single integer using ***state = row $\times$ 8 + column***, so the Start (row 0, col 0) is state 0 and the Goal (row 7, col 7) is state 63. The agent has four possible actions: Left (0), Down (1), Right (2), and Up (3). Attempting to move outside the grid simply leaves the agent in its current position.

Reward Structure

Reaching the Goal gives a reward of +1. Every other outcome (including falling into a Hole) gives a reward of 0. Holes and the Goal are both terminal states that end the episode immediately. Although falling into a Hole carries no explicit penalty, the episode-ending effect means the agent forfeits all future reward, which is sufficient to discourage that behavior during learning.

3. Q-Learning Algorithm

Q-Learning is a Reinforcement Learning algorithm that learns the value of taking a given action in a given state without needing to know the environment's rules in advance. It maintains a Q table of size 64×4 , where each entry $Q(s, a)$ estimates how good it is to take action (a) from state (s). After every step, the table is updated using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

Here α (the learning rate) controls how much new information overrides old estimates, r is the reward just received, γ (the discount factor) controls how much future rewards matter relative to immediate ones, and $\max Q(s', a')$ is the best estimated value achievable from the resulting next state. In effect, the agent nudges its current belief toward what it just experienced, scaled by the learning rate.

To balance exploring new actions against exploiting known good actions, the agent uses an epsilon-greedy strategy: with probability ϵ it picks a random action, and otherwise picks the action with the highest known Q-value. Epsilon starts at 1.0 and decays after every episode, gradually shifting the agent from exploration toward exploitation, with a minimum value preventing exploration from ever fully stopping.

4. Training Methodology

The table below summarizes the hyperparameters used, along with the reasoning behind each choice.

Hyperparameter	Value	Purpose	Why This Value
Learning rate (α)	0.1	Controls how quickly the Q values update based on new information	Standard default for Q Learning from; not exhaustively tuned.
Discount factor (γ)	99%	Gives strong weight to future rewards.	Deliberately chosen high because the only reward exists at the Goal; a high γ is needed for that reward to propagate backward and still be substantial across many cells or states during training
Initial epsilon	1	Agent starts by exploring purely at random	Standard default starting point. because the Q table is all zeros at the start, so there is nothing to exploit
Epsilon decay	0.9998	Controls how quickly exploration is phased out	Calculated analytically after debugging. An initial value of 0.995 caused epsilon to hit its low under 1000 episodes which was too early for the agent to realistically reach the Goal. 0.9998 was used to extend the exploration window further
Minimum epsilon	0.01	Ensures a small amount of exploration always remains	Standard default; something small to avoids the agent from becoming permanently stuck on a suboptimal policy late in training
Episodes	50,000	Number of training episodes	Originally an arbitrary starting choice of 10,000 was increased to 50,000 while debugging the epsilon decay issue. Increasing episodes alone did not fix the underlying bug, but 50,000 was retained because, once epsilon decay was corrected.
Max steps per episode	100	Prevents an episode from running indefinitely	Set generously above the minimum 14 steps needed to reach the goal, giving the agent room to wander during early random exploration without being cut off too abruptly

Debugging the Epsilon Decay Rate

An initial decay rate of 0.995 was tested first but caused training to fail with a 0% success rate even after increasing episodes from 10,000 to 50,000. Checks revealed that epsilon was reaching its minimum value under 1000 episodes which was too early for the agent to realistically reach the Goal at least once by random exploration. Once epsilon was decayed more slowly, the agent's longer exploration window allowed it to find the Goal early in training, after which the learned reward propagated through the Q table to improve the success rate.

Handling Random Variance Between Runs

Because exploration relies on randomness, an individual run can occasionally fail to find the Goal even once, purely by chance, even with the corrected decay rate. To guard against this, the training script automatically retries if a run completes with zero recorded successes, ensuring a usable, learned Q table is always produced.

5. Experimental Results

Figure 1 shows epsilon decaying smoothly from 1.0 toward its floor of 0.01, reaching that floor by approximately episode 20,000 as intended. Figure 2 shows the rolling success rate (averaged over the previous 500 episodes) climbing from 0% to approximately 98% by the end of training, with the steepest improvement occurring in the first 15,000 episodes — while epsilon was still high enough to keep exploring, but the Q-table had already begun to learn from early successes.

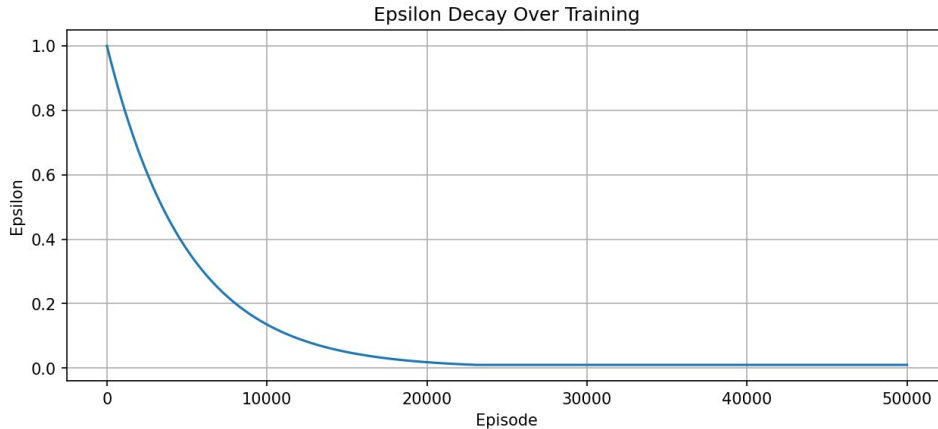


Figure 1. Epsilon decay over 50,000 training episodes.



Figure 2. Rolling success rate (500 - episode window) over training.

Following training, the agent was evaluated over 100 episodes using a purely greedy policy (no exploration). The results were:

- Success Rate: 100.00%
- Average Reward: 1.0000
- Number of Successful Runs: 100
- Number of Failures: 0

Because the environment's transitions are deterministic; every action always produces its intended movement; a policy that successfully avoids all holes will succeed on every evaluation episode with no variance. This explains the clean 100% result.

6. Learned Policy

The table below shows the action recommended by the trained Q-table for every state on the grid. Arrows indicate the best learned action; H marks a Hole; G marks the Goal.

```
[4]: %run evaluate.py
```

Learned Policy:

```

↓ ↓ ↓ ↓ ↓ ↓ ↓
→ → → → ↓ ↓ ↓
→ → ↑ H ↓ ↓ ↓
↑ ↑ ↑ H → ↓ ↓
↑ ↑ ↑ H → → ↓
↑ H H → → ↑ H ↓
↑ H ← ← H ↑ H ↓
← ← ← H ← ↑ ← G

```

```

Success Rate: 100.00%
Average Reward: 1.0000
Number of Successful Runs: 100
Number of Failures: 0

```

The learned route guides the agent down the left side of the grid, across to avoid the central column of holes, then down the right side before crossing safely to the Goal. Some arrows in squares far from this main route (for example, several Up arrows in the lower-left region) appear locally inconsistent. This is expected: once the agent converges on a working path, it rarely revisits states outside that path, so their Q-values remain less refined. This does not affect performance, since those states are never actually visited during evaluation.

7. Challenges Encountered

The most significant challenge was the epsilon decay bug described in Section 4, which initially produced a 0% success rate despite correct environment and agent logic. Diagnosing this required a structured process: first verifying the environment behaved correctly under pure random actions (confirming the Goal could be reached, just rarely), then checking the agent's action selection in isolation, before finally identifying that epsilon was decaying too quickly relative to how rare a chance success was. This was a valuable lesson in how exploration scheduling can silently undermine an otherwise correct implementation.

A secondary challenge was discovering that training outcomes can vary between runs of identical code due to randomness in exploration; an unlucky run could occasionally find zero successes even with a well-tuned decay rate. This was addressed by adding automatic retry logic to the training script.

8. Conclusion

This project implemented a complete Frozen Lake environment and Q Learning agent entirely from first principles, without relying on any existing Reinforcement Learning framework. After identifying and correcting an epsilon decay issue that initially prevented learning, the agent successfully converged to a near-optimal policy, achieving a 100% success rate over 100 evaluation episodes. The debugging process, in particular, reinforced a core practical lesson in Reinforcement Learning: the balance between exploration and exploitation must be tuned with respect to how rare positive reward signals are in a given environment, not chosen arbitrarily.